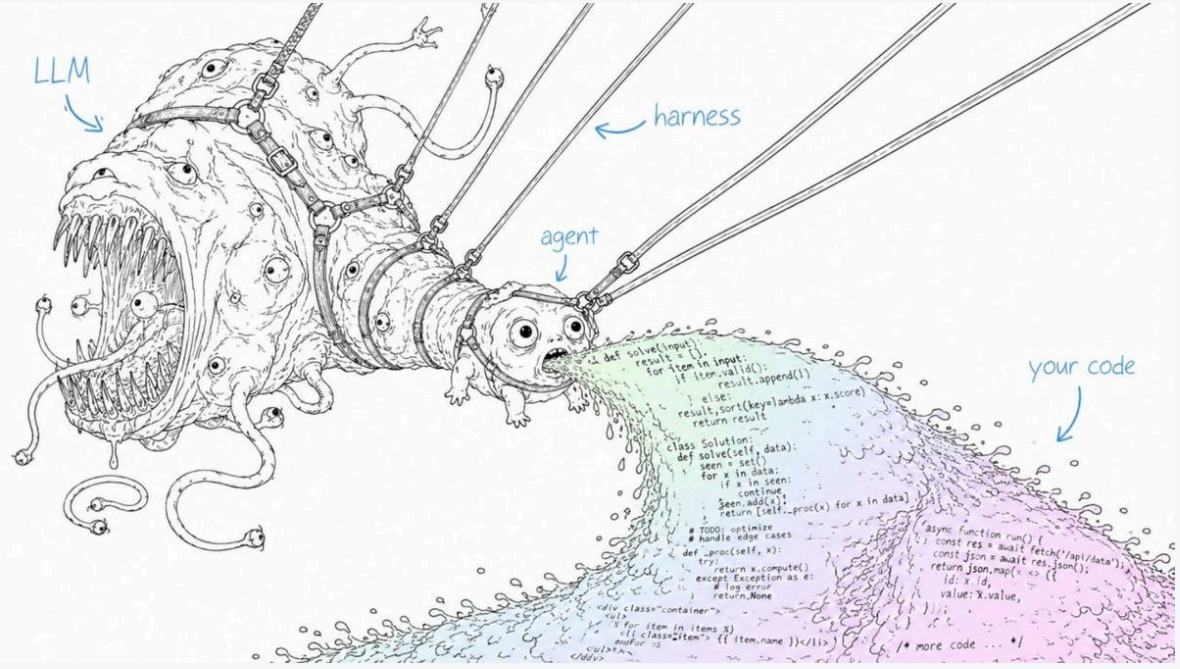
Agentic AI Workflows & Harness Engineering

Fundamentals & How to Build Your Own

Sergei Wallace



Audience and scope

For anyone who uses AI models for work, from coding agents to chat apps

- Main tools: Claude Code first, then Codex
- Most ideas also apply to chat and desktop apps
- My own harness is one example of what is possible

The conductor model

> “the AI engineer conducts an AI fleet using orchestrated workflows”
>
> The talk's meeting description

- You direct agents through workflows
- You adjust the harness so they need less direction
- Advanced: agents coordinate on long tasks; you handle decisions and blockers

Talk route

Every topic has its own pages on the site.

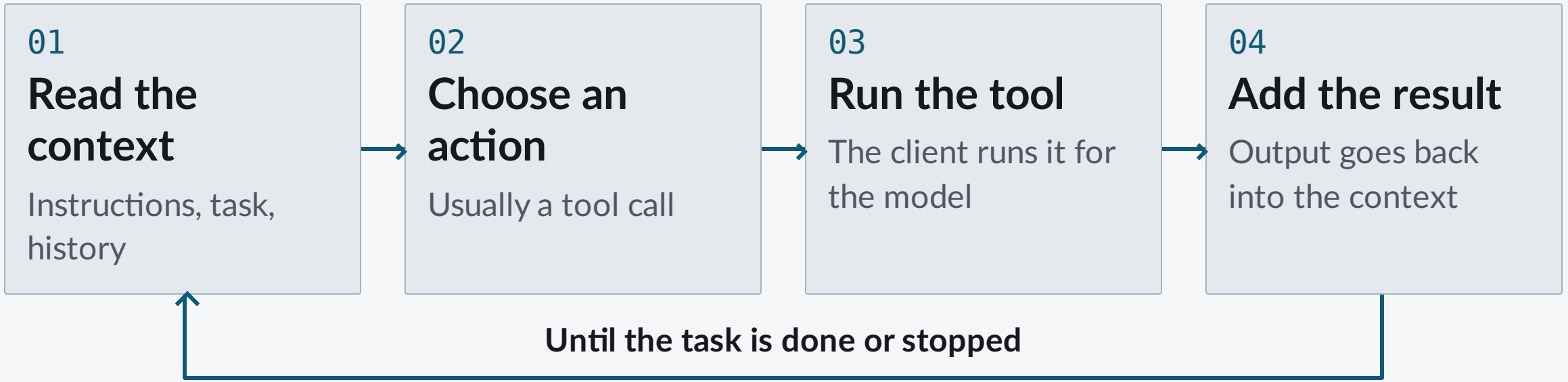
01 Agentic AI and harnesses

02 Building blocks in Claude Code and Codex

03 How the blocks combine

04 Building your own harness

The agent loop



- A chat model answers one prompt; an agent repeats this loop over many steps toward a goal

Context window and sessions

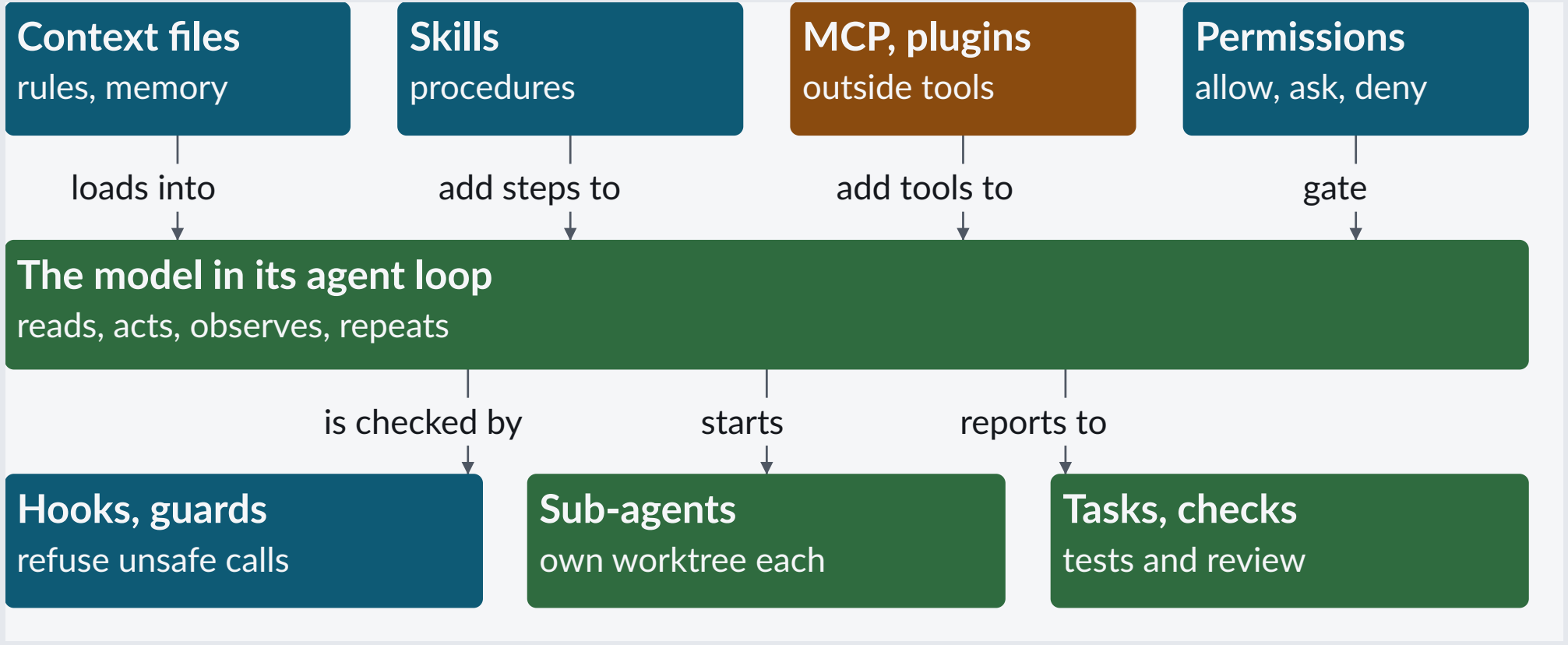
- Everything the model sees on one turn: instructions, conversation, tool results
- Nothing carries between sessions unless it is written to a file
- When an agent ignores a rule, check whether the rule was in its context

1M

tokens in the context window of several current Claude models

Parts of a harness

"Everything in an AI agent except the model itself" (Birgitta Böckeler)



Layout inspired by a community Claude Code architecture diagram

A systems approach to harness engineering

Each fix goes into the system; layers 2 to 4 enforce it.
Typed or prompted it twice? Change the harness, sooner if a repeat is costly.

01 Instructions: context files, skills and prompts

02 Workflows and tools that build the steps in

03 Hooks, scripts and linters that refuse or flag

04 Validation and tests before a change lands

Harness cost and effect

- Each part of a harness costs tokens on every turn
- Each part has to justify its cost

Task success: moved about 2 to 5 points across 21 model and harness pairs

Cost: Claude Code about twice a minimal four-tool harness on SWE-bench Lite

The building blocks

BLOCK	WHAT IT IS	MAIN USE
Context files	Instructions the client loads	Rules and project knowledge
Skills and plugins	Instructions that load on use	Repeatable procedures
Hooks and checks	Code at fixed points	Rules that must always hold
Templates and briefs	Fixed shapes for prompts	Consistent hand-offs
Sub-agents	Separate contexts	Delegated and parallel work

Claude Code and Codex

Claude Code

- `CLAUDE.md` context files
- Skills; sub-agents in `.claude/agents/`
- Hooks at 33 events
- Goal, loop and compact commands

Codex

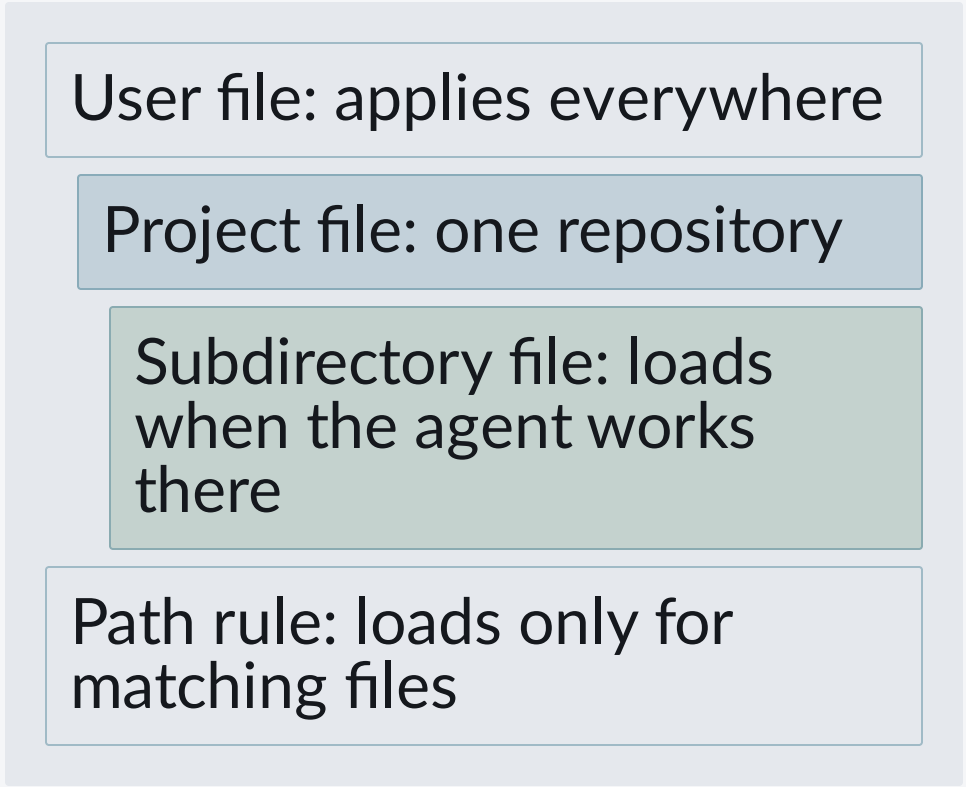
- `AGENTS.md` context files
- Skills; sub-agents as TOML in `.codex/agents/`
- Hooks at 14 events
- Goal and compact commands; loop not available

Native commands

COMMAND	WHAT IT DOES
/goal	Keep working until a condition is met
/loop	Repeat a prompt on an interval
/compact	Summarise to free context, with a focus
/rc	Control a session from another device; needs a claude.ai sign-in

Context files in practice

Keep the always-loaded file short: one line per rule, linking to a procedure (hub and spoke)



Skills and plugins

Skills

- A folder of instructions that loads only when used
- The model can start some; only the person can start others
- Person-only for side effects, such as publishing or merging

Plugins

- Skills, hooks, agents and MCP servers as one unit
- “Plugin module”: my name for one sharing a code library
- Each enabled plugin costs context on every turn

Templates and briefs

- Agents write most of a harness's prompts, so templates keep briefs consistent
- A sub-agent sees only its brief, so the rules it must follow go into the template
- The harness can also send text into a live session: a reminder after compaction, a wake-up, a message from another session

Enforcing rules in code

- Enforce as much as possible in code
- An agent follows written instructions less consistently as its context fills and its work piles up
- These run the same way every time:

Hook

Script or linter

Test

CI gate

Enforcement options

01
Context rule
Preferences and judgment calls

→

02
Path rule
Rules for some files only

→

03
Skill
Fixed steps, when it runs

→

04
Hook or guard
Checked when the agent acts

→

05
Test or CI gate
Checked at commit, push or merge

- Put each rule in the strongest step that can check it
- `/hooks` shows every configured hook and where it comes from, read-only

Process-kill guard case study

01
Rule in the brief
No kills by name pattern

→

02
An agent breaks it
Three unrelated test runs stop

→

03
A hook enforces it
Refused before it runs

→

04
Refusal says the fix
One process id or group

Hook refusal

```
$ pkill -f "pytest tests"
refusing to kill processes by command-line pattern. ...
Safe forms: kill a PID you captured yourself (...);
kill your own process group (`kill -- -<pgid>`, ...)
```

Cost of guards

- A check and a risky step

```
check          # fails
step           # runs anyway

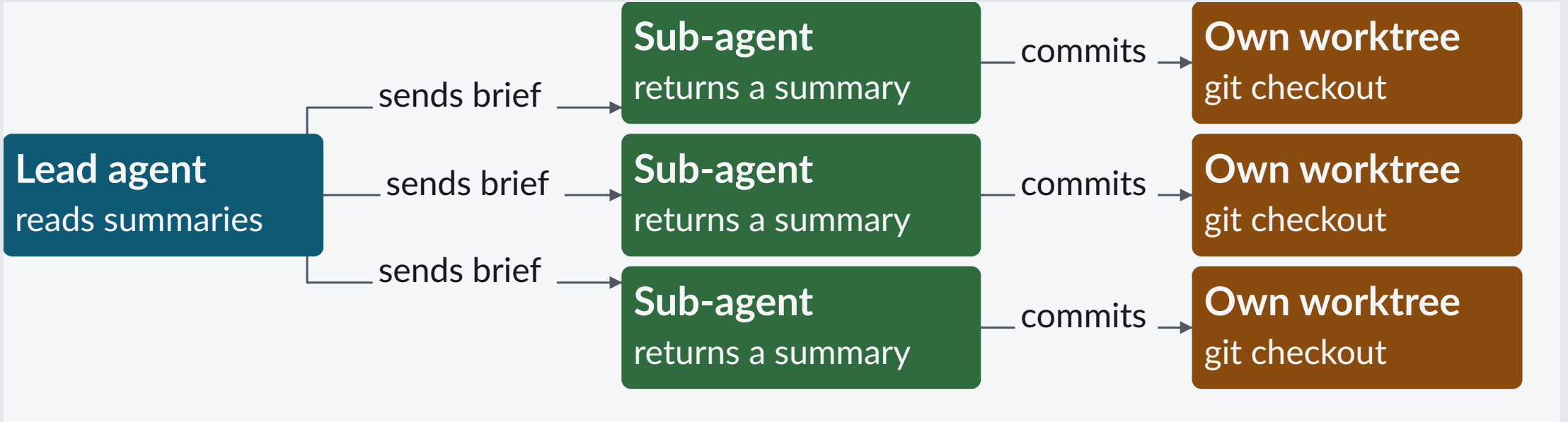
check && step
# step runs only if
# check passes
```

- A guard refuses what it cannot prove safe
- Guards need upkeep as the harness changes
- A hung hook can block all work
- Give each hook a time limit and a visible failure

Sub-agents and agent profiles

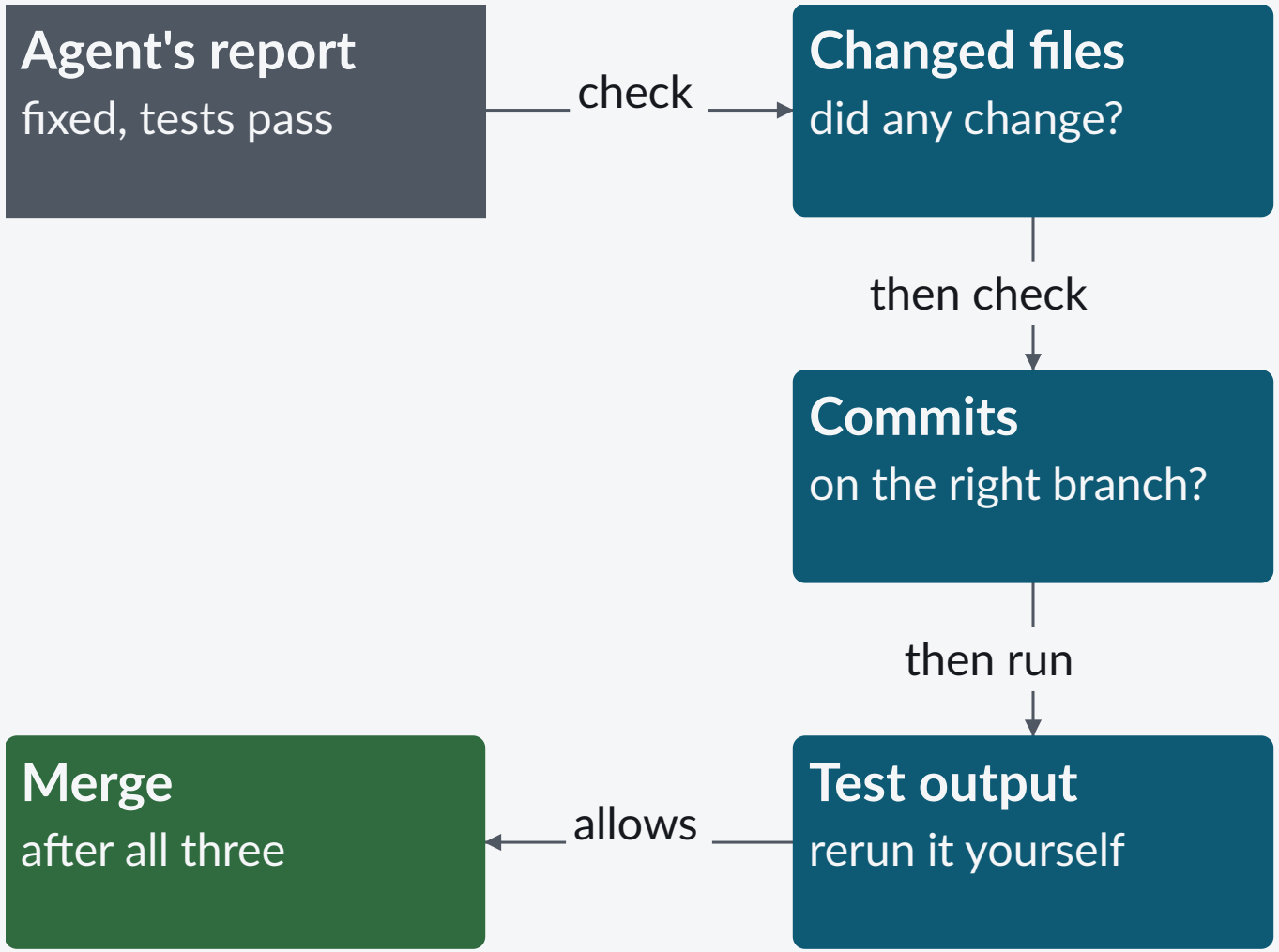
- A sub-agent works in its own context and returns a summary
- An agent profile pins a role's tools, model and permissions
- Pick the model per task: the strongest for decisions, cheaper ones for bulk reading
- Orchestration can be plain prompting: a lead session directs sub-agents through briefs and profiles

Delegation with worktrees



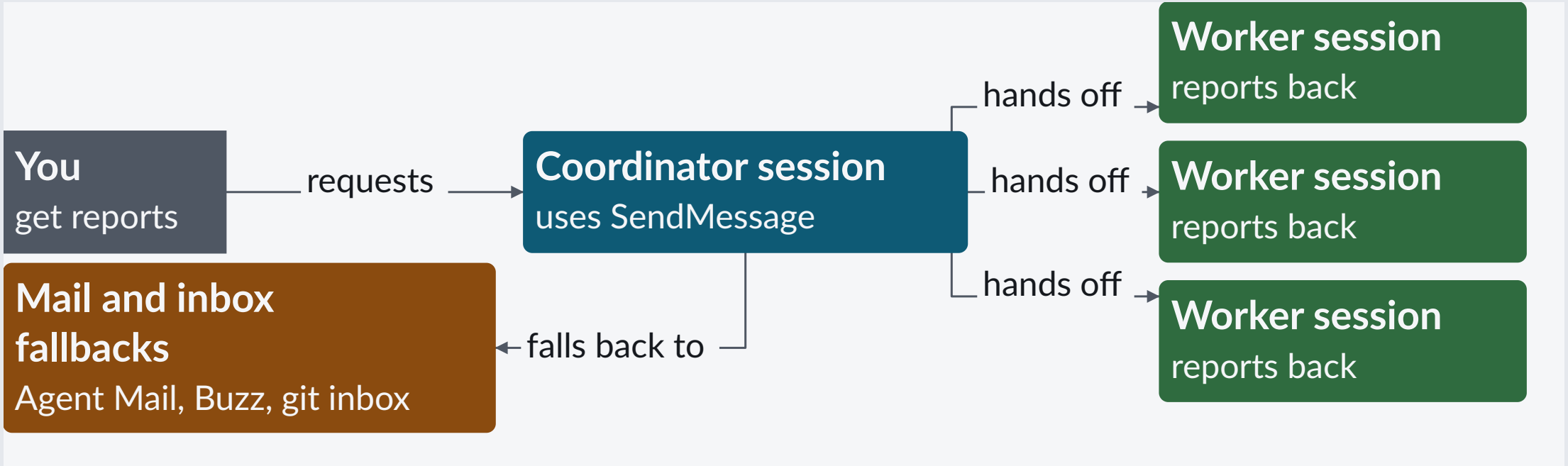
- Give each writing agent its own worktree; bound how many agents run at once and how deep they nest
- `/tasks` lists background work, finished sub-agents included
- `/subtask` starts a sub-agent that inherits the conversation; `/fork` copies the session to the background

Checking delegated work



- Agents can report work they did not do
- Read key passages, not only the summary

A coordinator and worker sessions



- A coordinator session gives you fewer places to deal with
- `ListAgents` and `/list-agents` show who a session can message

Two kinds of orchestration

Prompt-driven

- A lead agent directs others through briefs and profiles
- Fits work whose steps depend on judgment

Programmatic

- A script or graph holds the plan: dynamic workflows, LangGraph
- Fits known steps, parallel agents, resuming after failure
- `/workflows` watches, pauses and saves dynamic workflows
- Start one by asking, or with the `ultracode` keyword

Goals and loops

- Claude Code

```
/goal the test run prints
  no failures

/loop 5m check whether the
  deploy finished
```

- `/goal` runs until a small model judges it met
- `/loop` repeats a prompt on an interval
- Name printed evidence in the condition
- Give each loop an exit condition and a limit

Context window management

01
The context fills
Mistakes become more likely

→

02
Write state to files
Task, decisions, next step

→

03
Compact
A summary replaces the history

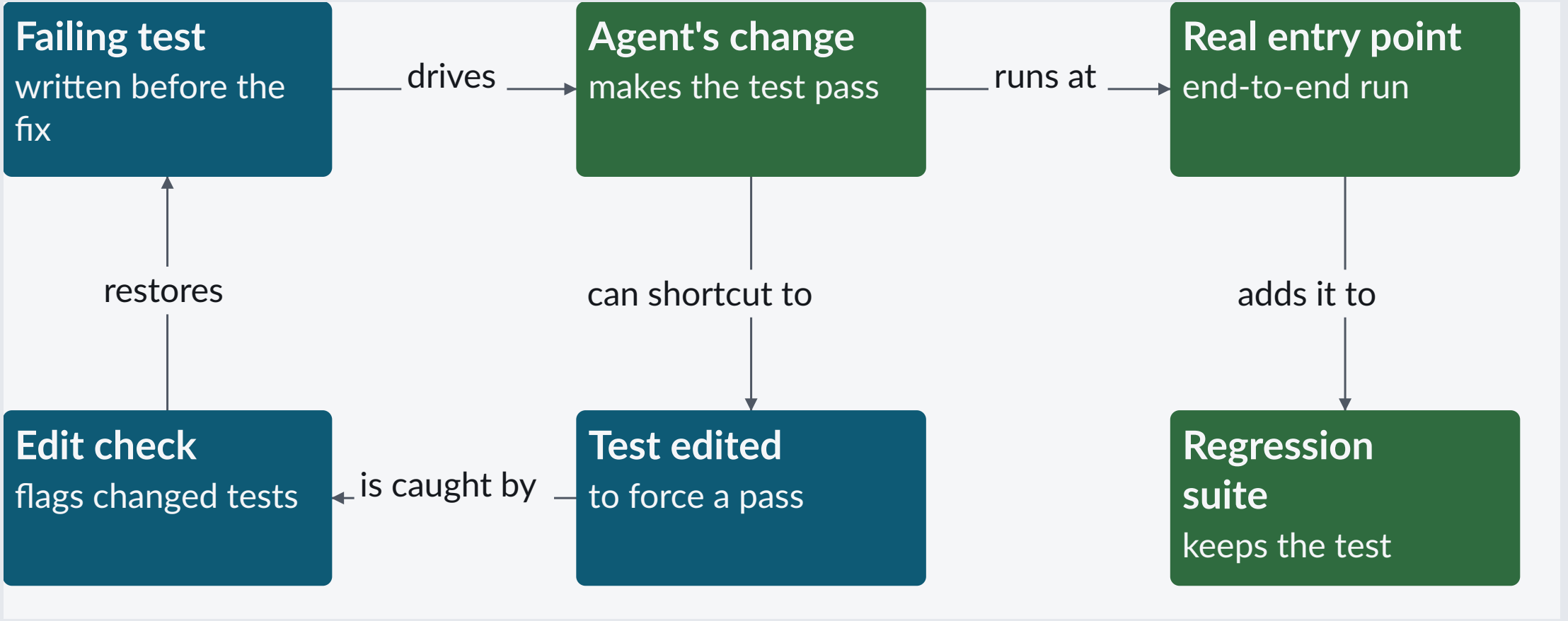
→

04
Resume from the files
A tracker holds the real task list

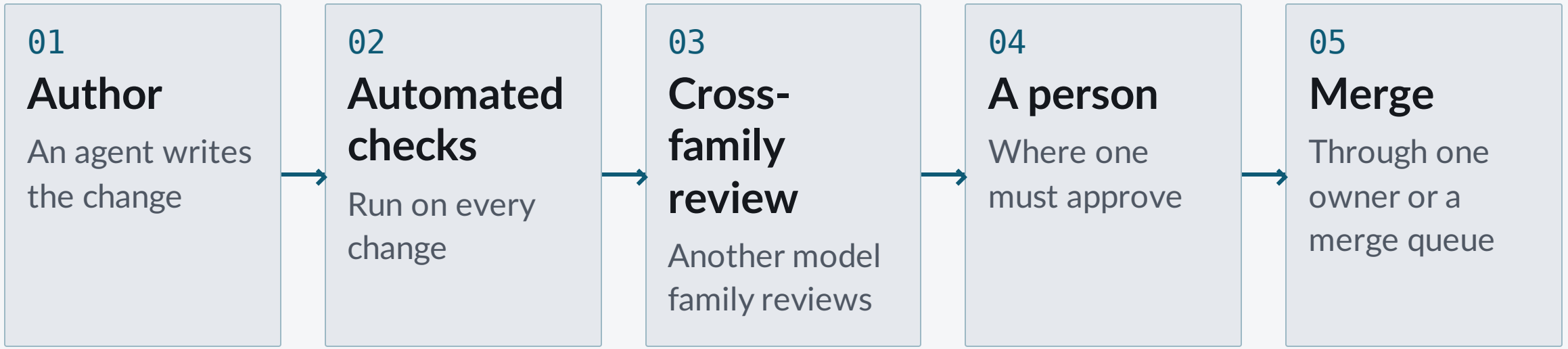
I compact at about 50 to 60 percent; the right point changes as models improve.

`/context` shows what is filling the context window, by category, with suggestions.

The test-first loop



Path of one change



- Have a separate agent verify each review finding: a reviewer asked to find problems usually reports some

Review practices

- A reviewer from another model family does not share the author's blind spots
- Fix the acceptance criteria before the review starts; stop when findings stop being new
- An agent's approval does not replace a person's where a person must approve
- `/code-review` (alias `/review`) and `/security-review` review a branch or pull request in a separate context

Open-ended review

> “A reviewer that’s asked to find problems will always find something. If you fix every finding, you end up with extra abstraction layers and tests for cases that can’t happen.”
>
> Claude Code best practices

- Say in the review brief that no change is a valid result
- I see it lead to overly defensive code

Human gates

Asking the person too often is also a failure: each question costs their time and stops the work.

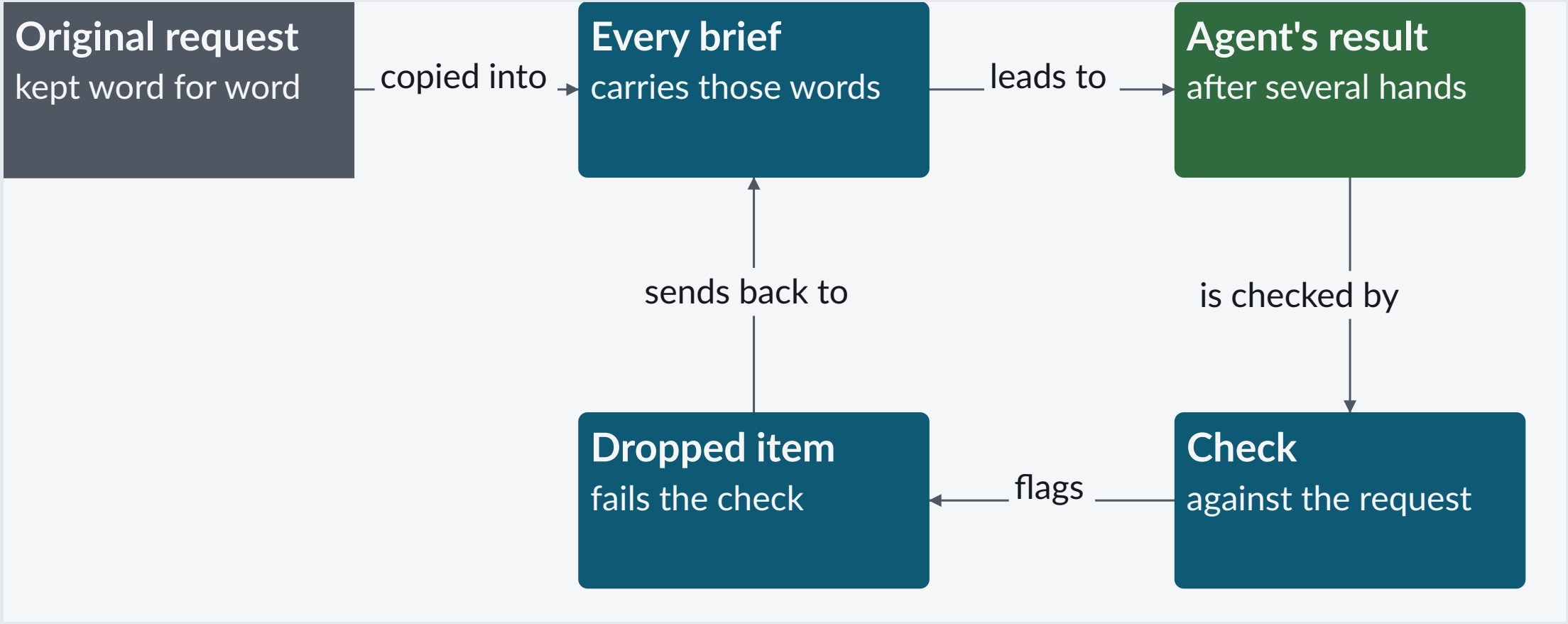
Needs a person

- Publishing, sending and deleting
- Spending money
- Architectural choices

The agent decides and records why

- Contained technical calls
- `/permissions` sets allow, ask and deny rules
- Auto mode lets a classifier approve routine actions in long runs

Alignment with the original request



A blocked agent investigates first, and asks only if a gate still applies

Evaluating harness changes

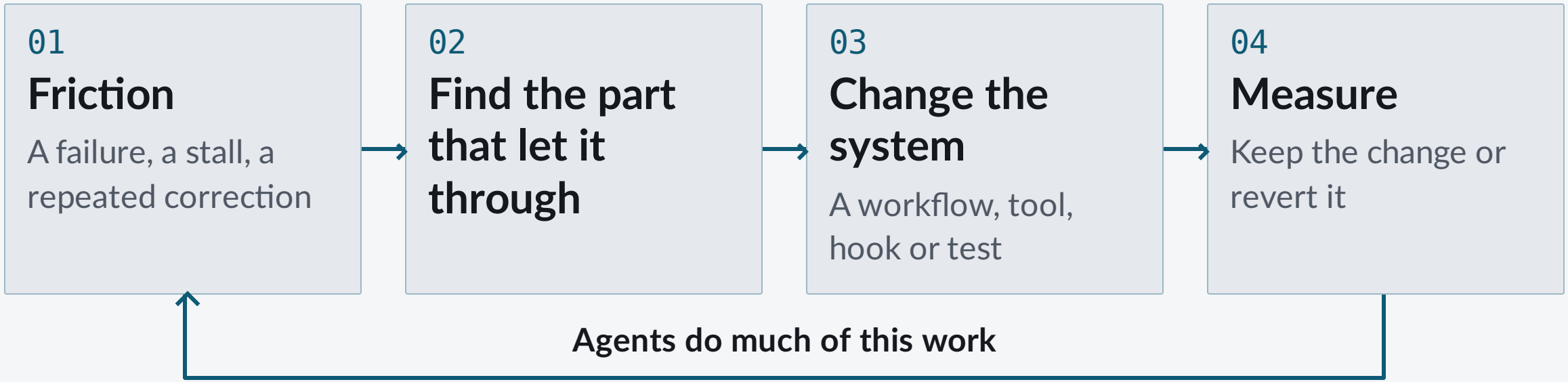
> “When we evaluate ‘an agent,’ we’re evaluating the harness and the model working together.”
>
> Anthropic, Demystifying evals for AI agents, 2026

Logging and evaluation

- Log commands run, files changed, time spent and cost
- Measure the harness itself; its bottlenecks are often somewhere unexpected
- `/insights` reports on your recent sessions, friction points included

- One change at a time, fixed task set, before and after
- Start with 20 to 50 real-failure tasks (Anthropic)

The improvement loop



Agents improving the harness

- An agent can file the bug, write the fix and add the check
- Skills and templates turn long instructions into short ones, for you and for agents
- Keep a person in the loop for outward, destructive and architectural decisions
- Studies of self-improving harnesses keep only changes that do not regress, with the evaluator outside what is changed

Complexity creep

How it happens

- Every fix adds a mechanism
- Rules, hooks and scripts overlap until nobody can follow them

What to do

- Merge rules and hooks that cover the same failure
- Retire mechanisms that compensated for older models
- Measure how often each fires and blocks correct work
- Measure the context each one uses

A possible first week

01
One CLAUDE.md
Rules and project knowledge

→

02
One skill
For the prompt you type most

→

03
One hook
For the rule an agent broke

→

04
One approval rule
What needs your approval

Model changes and trends

New models change what a harness needs.

- Re-test rules when the model changes
- Remove what no longer helps
- Agent Skills is a shared standard
- Codex can import a Claude Code setup

Questions and further reading

Recap of the conductor model: you direct the agents and adjust the harness so they need less direction

> - Write one context file this week
> - Turn the rule an agent has broken twice into a hook
> - Read the pages on the site

Appendix

0-3	4-7	8-11	12-15	16-19
Introduction	Context files	Delegation	Testing	Logging and evaluation
Agentic AI fundamentals	Skills and plugins	Session coordination	Code review	Building a harness
What is a harness?	Hooks and scripts	Orchestration	Human gates	Trends
Claude Code and Codex	Templates and stubs	Context window	Browser automation	Chat and desktop clients

Introduction

- From engineers running coding agents to people who use a chat app
- Applies to personal projects as well as to a job
- Advanced setups: agents coordinate, and you step in on decisions

Three depths

01 Talk slide

02 Overview page

03 Topic pages

Agentic AI fundamentals

An agent works toward a goal over many steps.

- Tools let it act: files, commands, search, services
- The context window is all it sees on one turn, up to one million tokens
- No memory between sessions unless written to files

What is a harness?

> “harness has emerged as a shorthand to mean everything in an AI agent except the model itself”
>
> Birgitta Böckeler, Thoughtworks, 2026

- Guides steer the agent before it acts
- Sensors give it feedback after it acts
- Some parts are instructions, others code
- Each loaded part costs tokens every turn

Claude Code and Codex basics

NEEDS A CLAUDE.AI SIGN-IN
Remote Control
Claude in Chrome
Computer use

Codex

- Hooks, skills and TOML sub-agents
- Goal, compact and memories commands; loop not available
- `/import` reads a Claude Code or Cursor setup

Context files

- Imports with `@path`, up to four levels deep
- Codex reads one `AGENTS.md` per directory, up to 32 KiB
- Claude Code reads `AGENTS.md` only when no `CLAUDE.md` exists
- Keep each `CLAUDE.md` under 200 lines

`.claude/rules/tests.md`

```
---
paths:
  - "tests/**"
---
Rules for test files
```

Skills, commands and plugins

FRONTMATTER	WHO CAN START IT
Default	You and the model
disable-model-invocation: true	Only you
user-invocable: false	Only the model
Codex: allow_implicit_invocation: false	Only you

- Plugins bundle skills, hooks, agents, MCP servers
- Up to six skills in one message
- Delete skills nobody invokes

Hooks, scripts and linters

> “stop, don’t do this”
>
> Claude Code hooks reference, on exit code 2

01 Context rule

02 Path rule

03 Skill

04 Hook or guard

05 Test or CI gate

Templates and stubs

Sub-agent brief stub

```
Objective
Inputs
Output format, length cap
Tools and scope
Boundaries
When stuck
```

- A stub is a fill-in-the-blanks brief
- A template is the full shape of one kind of document
- The brief carries the safety rules word for word

Agent profiles, sub-agents and delegation

TASK	MODEL
Decisions, synthesis	Strongest
Bulk reading, edits	Cheaper

3

levels of sub-agent nesting by default in Claude Code; a setting changes it

Coordinating several sessions

- One session per machine can own installs and pushes
- Check each channel can reach its target before use

> ### Message order in my harness
>
> 01 SendMessage
>
> 02 Agent Mail
>
> 03 Buzz
>
> 04 A git inbox for other machines

Orchestration and workflows

FRAMEWORKS IN USE
LangGraph
OpenAI Agents SDK
CrewAI
Google Agent Development Kit
Microsoft Agent Framework

- Workflow runs: 16 agents at once, 1,000 total
- Hill-climbing keeps only measured gains

Context window and memory

Loaded again after compaction

- Root CLAUDE.md
  - Rules without paths
    - Auto memory

I compact at about 50 to 60 percent.

Others report a sweet spot below about 300k tokens (unverified).

Codex compacts near 250k tokens by default, by my observation (unverified).

Testing agent-written code

5.3%

test tampering across eight models, down from 23.6%, with a policy line and a way to report a bad test

- Run affected tests; full suite less often
- A project deleted 400,000 lines of its own tests
- Its code coverage barely moved
- Keep every gate passable wherever it runs

Code review

- Route merges through one owner or a merge queue
- Run the affected tests on the merged result
- Review briefs say that no change is a valid result

> “An agent’s approval never counts as a person’s.”
>
> Claude Code team, Solving code review at Anthropic

Appendix Backup

Human gates, decisions and alignment

Needs a person

Publishing, sending, deleting, spending money, architectural choices

The agent decides

Contained technical calls, with a record of why

When blocked

A blocked agent investigates first, and asks only if a gate still applies

Browser automation

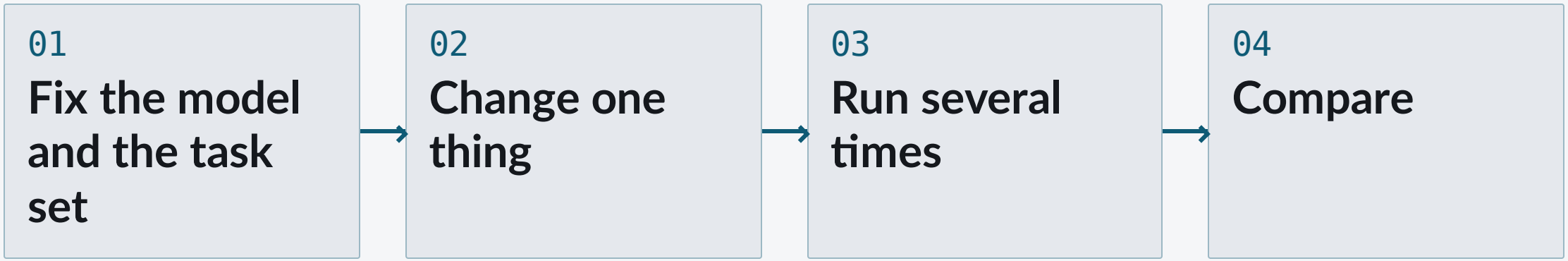
Treat web page content as untrusted.

ROUTE	AVAILABILITY
Claude in Chrome	Direct Anthropic plan; not through Amazon Bedrock
Computer use	Research preview on macOS, Pro, Max and Ultra
CDP or Playwright	Any provider

Appendix Backup

Logging, observability and evaluation

- Put a budget check before any job that makes paid model calls
- `claude plugin eval` compares results with and without a plugin
- About 30% of one benchmark's tasks were found broken (not yet verified)



Building your own harness

> Automatic harness evolution “fails to outperform simple test-time scaling methods”
>
> arXiv 2607.12227, 2026

- **Mine failures**
- **Make small bounded edits**
- **Keep only changes that do not regress**

Trends and model changes

> “The bottleneck, now, is all of the developer infrastructure built for human scale”
>
> Anthropic, 2026

30,000

and more code changes merged per day at Anthropic as of September 2026, up from about 2,500 a day in May

Chat, desktop and web clients

CARRIES OVER	ADAPTED	DOES NOT APPLY
Desktop and web Claude Code	Chat projects play the role of context files (unverified)	Hooks, scripts and local guards
Notes, the original request, checks, approvals	Plugins on claude.ai and in Cowork	Local plugins in a cloud session

Mods and function hooks

A mod is a plugin whose hooks module registers function hooks on tool calls, prompts and turns.

Shell hook

- A shell script
- A new process per event
- Cannot draw
- Guards and checks

Function hook

- JavaScript or TypeScript
- Functions loaded into the client
- Can also draw panes, status line and toasts
- Guards, behaviour changes, interface changes